Reviewer Pack — Minimal Order of Monomial Ideals Bounds Communication Comple…

Entry b7332854b8fa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 21:54:46 UTC

Minimal Order of Monomial Ideals Bounds Communication Complexity for XOR

Entry ID: `b7332854b8fa` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 21:54:46 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Monomial Ideals) **Field B** (complexity object): Boolean Function Complexity

Statement:

> For a given XOR Boolean function f, the minimal order of a monomial ideal I that contains the vanishing ideal of f is upper-bounded by the degree of f. Formally, for all XOR Boolean functions f: {0,1}^n -> {0,1}, there exists a constant c such that |I| ≤ deg(f) * c.

Rationale (proposer's reasoning):

> Monomial ideals are closely related to Gröbner bases, which have been used in studying algebraic complexity. The conjecture aims to bridge the gap between algebraic and boolean functions by relating the order of monomial ideals to communication complexity for XOR Boolean functions. This could provide new insights into understanding the inherent difficulty of certain computational problems.

Taxonomy category: `cg_kw_andreev` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0f1550dc4ba70d38`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean of the order |I| divided by the degree deg(f) across 30 seeds is less than or equal to a constant c, i.e., 'mean(|I|/deg(f)) ≤ c'. The conjecture is falsified if any seed produces an order |I| greater than the degree deg(f) * c.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	SAFE	HITS
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"monomial ideal" AND "communication complexity" AND "XOR function"` - `"vanishing ideal" IN BOOLEAN FUNCTION COMPLEXITY" AND monomial ideal"` - `"degree of XOR function" AND upper bound ON minimal order OF monomial ideal"`

Top relevant hits considered: - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1501.06495v3] On operator algebras associated with monomial ideals in noncommuting variables - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5 + random.randint(0, 34)  # Ensure n_min >= 5 and n_max >= 20
    if n > 40:
        return {
            "metric_name": "order_div_degree",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "n_too_large"
        }

    # Generate a random XOR Boolean function f with n variables
    f = [random.choice([0, 1]) for _ in range(2**n)]

    # Compute the vanishing ideal I of f
    I = set()
    for i in range(2**n):
        if f[i] == 1:
            monomial = [i >> j & 1 for j in range(n)]
            I.add(tuple(monomial))

    # Compute the order |I| and the degree deg(f) of f
    order_I = len(I)
    degree_f = n
```

```
    # Check if |I| ≤ deg(f) * c for a constant c
    c = 2  # Example constant, can be adjusted as needed
    conjecture_holds = order_I <= degree_f * c

    return {
        "metric_name": "order_div_degree",
        "metric_value": order_I / degree_f,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"order_I={order_I}, deg_f={degree_f}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print(f"RESULT: INCONCLUSIVE budget_exceeded n_tested={len(results)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means we cannot verify the mean of |I|/deg(f) across the seeds. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17516
2	preregistration	ollama_remote	glm4:latest	0	0	9355
3	novelty	ollama_remote	glm4:latest	0	0	15316
4	novelty	ollama_remote	glm4:latest	0	0	9301
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13038
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6935
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8625
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14039
9	judge	ollama_remote	glm4:latest	0	0	23615

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 117741 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b7332854b8fa.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b7332854b8fa.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b7332854b8fa.tar.gz` (if generated)